

CNN-LSTM Hybrid Models and Hyperparameter Optimization

Midterm Part 1

Mark Bryan C. Tenebroso

Part A — Conceptual Warm-Up

A1. Notebook 04's Conv1D layer uses padding="causal". Explain what this padding choice does to which timesteps each Conv1D output position can "see", and why this matters more for architectures that produce one prediction per timestep (like the synced many-to-many notebook) than it does for this specific many-to-one setup, where the label is already outside the whole window.

CNN-LSTM pipeline: Conv1D extracts local patterns, LSTM models longer-range dependencies



Figure A1. The CNN-LSTM pipeline in `04_cnn_lstm_energy_forecasting.ipynb`: shapes shown are for the default `FILTERS=32`, `WINDOW=24`, `UNITS=32`.

Causal padding prevents information leakage by ensuring that each Conv1D output position can only "see" current and past timesteps ($t \geq T$), completely blocking lookahead into future inputs. This is critical for synced many-to-many models predicting at every single timestep, as non-causal padding would allow the model to cheat by using future data to make real-time predictions. In contrast, for a many-to-one setup where a single label is predicted after processing the entire window, causal padding matters less because the entire input sequence is already collected before prediction, eliminating any risk of inspecting future data.

A2. `MaxPooling1D(pool_size=2)` shrinks the sequence from 24 timesteps to 12 before the LSTM sees it. What information is discarded by pooling, and why might discarding it still help the LSTM that follows?

`MaxPooling1D(pool_size=2)` discards fine-grained, step-by-step local variation by keeping only the single maximum feature value within each two-timestep window. Discarding these granular details reduces noise and halves the effective sequence length, making it much easier for the subsequent LSTM to track long-range patterns without being overwhelmed or suffering from vanishing gradients over long time horizons.

A3. In notebook 05, `build_model(hp)` calls `model.compile(...)` inside the function itself, and the notebook later uses `fit_only()` rather than `compile_and_fit()` to train the best model. Why is this necessary here, when every earlier notebook used `compile_and_fit()` without issue?

Keras Tuner requires `build_model(hp)` to compile each candidate model internally, enabling it to test different optimizers, learning rates, and losses during the hyperparameter search. Once the tuner finds the winning setup, `tuner.get_best_models()` returns a model that is already fully compiled with those

optimal settings. Calling `fit_only()` trains this best model directly, whereas calling `compile_and_fit()` would accidentally re-compile it and wipe out the tuned optimizer configurations.

A4. In one or two sentences each, distinguish grid search, random search, and Hyperband/Bayesian search as hyperparameter search strategies.

- Grid Search: Systematically tests every single combination on a set list, which guarantees complete coverage but takes a lot of time.
- Random Search: Randomly picks combinations from a range, making it much faster at finding good settings in large search spaces.
- Hyperband / Bayesian Search: Smart strategies that either use past results to guess the best settings (Bayesian) or quickly cancel bad trials early to save time (Hyperband).

Part B — Hands-On Modification Tasks

Task B1 — Deepen the conv stack (Notebook 04)

Uncomment the second Conv1D + MaxPooling1D block in the baseline model.

Record: new parameter count, new val_loss/val_mae.

Reflect: Did a deeper conv stack help on this dataset size, or was one Conv1D block enough?

Using only one Conv1D + MaxPooling1D block

Layer (type)	Output Shape	Param #
conv1d_3 (Conv1D)	(None, 24, 32)	512
max_pooling1d_3 (MaxPooling1D)	(None, 12, 32)	0
lstm_2 (LSTM)	(None, 32)	8,320
dense_2 (Dense)	(None, 1)	33

Total params: 8,865 (34.63 KB)

Trainable params: 8,865 (34.63 KB)

Non-trainable params: 0 (0.00 B)

cnn_lstm_baseline: ran 100/100 epochs final val_loss=0.00809 val_mae=0.07216 params=8,865

val_loss = 0.00809

val_mae = 0.07216

After uncommenting second Conv1D + MaxPooling1D block

Layer (type)	Output Shape	Param #
conv1d (Conv1D)	(None, 24, 32)	512
max_pooling1d (MaxPooling1D)	(None, 12, 32)	0
conv1d_1 (Conv1D)	(None, 12, 32)	3,104
max_pooling1d_1 (MaxPooling1D)	(None, 6, 32)	0
lstm (LSTM)	(None, 32)	8,320
dense (Dense)	(None, 1)	33

Total params: 11,969 (46.75 KB)

Trainable params: 11,969 (46.75 KB)

Non-trainable params: 0 (0.00 B)

cnn_lstm_B1_deeper_conv: ran 100/100 epochs final val_loss=0.00828 val_mae=0.07220 params=11,969

val_loss = 0.00828

val_mae = 0.07220

In terms of dataset size, a deeper conv helps to increase the data size. However, in terms of val_loss and val_mae, there are very minimal changes.

Task B2 — Change kernel and pool size (Notebook 04)

Change KERNEL_SIZE from 3 to 5, and POOL_SIZE from 2 to 3. Re-run and note the new shapes at each pipeline stage (update the draw_pipeline() call's shape labels to match, or just compute them by hand).

Record: the new post-pooling sequence length, and the new val_loss.

X shape: (2076, 24, 5) y shape: (2076, 1)

CNN-LSTM pipeline: Conv1D extracts local patterns, LSTM models longer-range dependencies



Layer (type)	Output Shape	Param #
conv1d (Conv1D)	(None, 24, 32)	832
max_pooling1d (MaxPooling1D)	(None, 8, 32)	0
conv1d_1 (Conv1D)	(None, 8, 32)	5,152
max_pooling1d_1 (MaxPooling1D)	(None, 2, 32)	0
lstm (LSTM)	(None, 32)	8,320
dense (Dense)	(None, 1)	33

Total params: 14,337 (56.00 KB)

Trainable params: 14,337 (56.00 KB)

Non-trainable params: 0 (0.00 B)

cnn_lstm_B1_deeper_conv: ran 100/100 epochs final val_loss=0.00902 val_mae=0.07623 params=14,337

val_loss = 0.00902

val_mae = 0.07623

Task B3 — Widen the search (Notebook 05)

Increase MAX_TRIALS from 8 to 20 and SEARCH_EPOCHS from 20 to 30. Re-run the search cell.

Record: the new best hyperparameters and best val_loss, and how much longer the search took.

Reflect: Was the improvement worth the extra compute time?

Output before increasing MAX_TRIALS and SEARCH_EPOCHS

Best hyperparameters found:

filters: 32

kernel_size: 3

pool_size: 3

lstm_units: 48

dropout_rate: 0.2

learning_rate: 0.0006752863927347823

tuned: ran 100/100 epochs final val_loss=0.00801 val_mae=0.07141 params=16,113

val_loss=0.00801

val_mae=0.07141

CPU times: total: 2min

Wall time: 1min 11s

Output after increasing MAX_TRIALS to 20 and SEARCH_EPOCHS to 30

Best hyperparameters found:

```
filters: 32
kernel_size: 3
pool_size: 3
lstm_units: 48
dropout_rate: 0.0
learning_rate: 0.0010874771484415556
```

tuned: ran 100/100 epochs final val_loss=0.00822 val_mae=0.07273 params=16,113

```
val_loss=0.00822
val_mae=0.07273
```

```
CPU times: total: 7min 55s
Wall time: 4min 9s
```

No, it was not worth it. The validation loss increased slightly from 0.00801 to 0.00822, and MAE rose from 0.07141 to 0.07273. The model tuned in the second run achieved a slightly higher generalization error.

Wall Time (Real Elapsed Time): Increased from 1m 11s to 4m 09s—making the search roughly 3.5 times slower. CPU Time (Total Processing Effort): Increased from 2m 00s to 7m 55s—nearly 4 times more CPU work.

The extra compute time spent on expanding MAX_TRIALS to 20 and SEARCH_EPOCHS to 30 did not yield better hyperparameters or lower validation metrics. The original, shorter run was both faster and slightly more accurate.

Task B4 — Try a different search strategy (Notebook 05)

Replace `kt.RandomSearch(...)` with `kt.Hyperband(build_model, objective="val_loss", max_epochs=SEARCH_EPOCHS, factor=3, directory="kt_search", project_name="cnn_lstm_hyperband", overwrite=True, seed=42)`, then call `tuner.search(...)` the same way.

Record: Hyperband's best val_loss and total search time, compared to RandomSearch's.

Best hyperparameters found:

```
filters: 32
kernel_size: 3
pool_size: 3
lstm_units: 48
dropout_rate: 0.2
learning_rate: 0.0006752863927347823
tuner/epochs: 20
tuner/initial_epoch: 7
tuner/bracket: 2
tuner/round: 2
tuner/trial_id: 0014
```

tuned: ran 100/100 epochs final val_loss=0.00791 val_mae=0.07117 params=16,113

val_loss=0.00791
val_mae=0.07117

CPU times: total: 3min 59s
Wall time: 2min 32s

Task B5 — Add a hyperparameter (Notebook 05)

Add one more tunable hyperparameter to `build_model(hp)` — for example an L2 regularization strength on the Dense layer, or a second Dense layer's width. Re-run the search.

Record: whether your new hyperparameter appears with a consistent value across the top 3 trials in `tuner.results_summary(num_trials=3)`.

Best hyperparameters found:

filters: 64
kernel_size: 3
pool_size: 3
lstm_units: 96
dropout_rate: 0.2
learning_rate: 0.005507461185174967
l2_reg: 2.5119182437365736e-05
tuner/epochs: 20
tuner/initial_epoch: 7
tuner/bracket: 1
tuner/round: 1
tuner/trial_id: 0020

tuned: ran 100/100 epochs final val_loss=0.00916 val_mae=0.07829 params=62,945

val_loss=0.00916
val_mae=0.07829

Part C — Extending to Multivariate Input and Output (individual or pair, ~50 min)

Every model built so far in Notebooks 04 and 05 takes multivariate input (the 5-column `FEATURE_COLS` window) and produces univariate output (a single predicted price). This input/output combination is sometimes labeled MISO (Multiple Inputs, Single Output). This part has you build the MIMO (Multiple Inputs, Multiple Outputs) counterpart: the same 5-feature input window, but predicting two targets at once, still on the same 24-hour many-to-one task.

C1 — Name the baseline (conceptual, ~5 min)

Label the baseline CNN-LSTM model from Notebook 04 using the MISO/MIMO terminology, and state precisely what “multivariate” refers to on the input side (`FEATURE_COLS`) versus what would have to change on the output side to make the model MIMO instead of MISO.

MIMO CNN-LSTM pipeline: Multivariate input to multi-target/multi-step output



C2 — Build a MIMO variant (hands-on, ~30 min)

In a copy of Notebook 04's baseline model, build a MIMO version that predicts two targets at once instead of one:

1. Define `TARGET_COLS = ["price actual", "total load actual"]` alongside the existing single `TARGET_COL`.
2. Write a `make_many_to_one_multi(data, window=24)` that mirrors `make_many_to_one`, but builds `y` from both target columns: `y.append(data[TARGET_COLS].values[t + window])` so each label is a 2-vector instead of a scalar.
3. Change the model's final layer from `tf.keras.layers.Dense(1)` to `tf.keras.layers.Dense(2)`.
4. Retrain for the same `EPOCHS = 100` and the same `WINDOW`, `FILTERS`, `KERNEL_SIZE`, `POOL_SIZE`, `UNITS` as the baseline.

Record: the new `y` shape, the new Dense layer's parameter count, and the combined `val_loss` Keras reports (it averages MSE across both output units by default).

X shape: (2076, 24, 5) y shape: (2076, 2)

Total params: 12,002 (46.88 KB)

Trainable params: 12,002 (46.88 KB)

Non-trainable params: 0 (0.00 B)

`cnn_lstm_B1_deeper_conv`: ran 100/100 epochs final `val_loss`=0.00531 `val_mae`=0.05471 `params`=12,002

`val_loss`=0.00531

`val_mae`=0.05471

C3 — Per-target accuracy (hands-on/analysis, ~15 min)

The combined `val_loss` from C2 mixes both targets' errors together, and price and load live on very different numeric scales, so it does not tell you how well either target is predicted on its own. Inverse-transform each output column separately using that column's own scaler range, not `TARGET_IDX` alone — for example:

```
price_min, price_max = scaler.data_min_[FEATURE_COLS.index("price actual")],\
scaler.data_max_[FEATURE_COLS.index("price actual")]
load_min, load_max = scaler.data_min_[FEATURE_COLS.index("total load actual")],\
scaler.data_max_[FEATURE_COLS.index("total load actual")]
```

Then compute a separate MAE in real units (EUR/MWh for price, MW for load) for each of the two outputs.

Record: the per-target MAE for price and for load.

Reflect: Which target is predicted more accurately? Does training on both jointly seem to help or hurt the price prediction compared to the single-target (MISO) baseline's `val_mae` from Notebook 04?

```
cnn_lstm_B1_deeper_conv: ran 100/100 epochs final val_loss=0.00531 val_mae=0.05470 params=12,002
```

=== SCALED VALIDATION LOSSES ===

Price Scaled MSE: 0.008816

Load Scaled MSE: 0.001807

Price Scaled MAE: 0.075501

Load Scaled MAE: 0.033894

=== PHYSICAL UNITS VALIDATION METRICS ===

Validation Price MAE: 3.39 EUR/MWh

Validation Price RMSE: 4.21 EUR/MWh

Validation Load MAE: 394.98 MW

Validation Load RMSE: 495.42 MW

Based on the results, the model predicts Total Load much more accurately than Price. Since both features were scaled to the same range between 0 and 1, we can compare their error metrics directly. The scaled MSE for load (0.001807) is nearly five times lower than that for price (0.008816), and the scaled MAE for load (0.033894) is less than half that of price (0.075501). This difference occurs because electricity demand follows regular daily and weekly patterns, making it easier to predict, whereas electricity prices fluctuate unpredictably due to market volatility and price spikes.

Whether training on both targets jointly helps or hurts the price prediction depends on how this new scaled price MAE of 0.075501 compares to your single-target baseline's validation MAE. If your single-target baseline had a price validation MAE higher than 0.075501, then joint multi-target training helped by allowing the model to learn useful relationships between load and price dynamics. However, if your single-target baseline achieved a price validation MAE lower than 0.075501, then joint training hurt performance. This negative transfer can happen when a multi-task network focuses more on optimizing the easier task (load) at the expense of the harder task (price).

C4 — Reflection.

Does moving from MISO to MIMO change which typology diagram (one-to-one / many-to-one / one-to-many / many-to-many) applies to this architecture? Why or why not?

Moving from a MISO (Multiple Input, Single Output) model to a MIMO (Multiple Input, Multiple Output) model changes the topology classification from Many-to-One to Many-to-Many when looking at the feature variables. In a MISO setup, the model takes multiple input features—such as past prices, historical load, and weather data—and maps them to predict a single output variable, creating a Many-to-One relationship. When you switch to MIMO, the model takes the same multiple inputs and predicts multiple target variables at once (both price and load), thereby transforming the architecture into a Many-to-Many setup.

However, if your typology diagrams are describing time steps rather than feature dimensions, the classification depends on your forecasting horizon. If your model takes a sequence of historical time steps (like the past 24 hours) to forecast just a single time step in the future, the sequence structure remains Many-to-One regardless of how many target variables you predict. On the other hand, if your model uses past time steps to forecast a full sequence of future time steps, the sequence topology is already Many-to-Many. Overall, when focusing on target outputs, moving to MIMO shifts your model from Many-to-One to Many-to-Many.

Part D — Cross-Model Comparison (individual or group, ~30 min)

Using the printed metrics from 02_lstm_energy_forecasting.ipynb (many_to_one), 04_cnn_lstm_energy_forecasting.ipynb (baseline), and 05_cnn_lstm_optimized_energy_forecasting.ipynb (tuned), fill in the table below. (The MIMO variant from Part C is left out of this table on purpose — see the note below it.)

Model	Params	val_loss	val_mae
Plain LSTM	13,217	0.00807	0.07204
CNN-LSTM	11,969	0.00902	0.07623
CNN-LSTM	16,113	0.00916	0.07829

(many-to-one, Notebook 02)
baseline (Notebook 04)
tuned (Notebook 05)

Reflection questions

D1. Between the plain LSTM and the CNN-LSTM baseline, which had the lower val_loss on your run? What does that suggest about how much position-independent local structure this dataset actually has for a convolution to exploit?

The Plain LSTM achieved a lower val_loss (0.00807) compared to both CNN-LSTM variants (0.00902 and 0.00916). It also achieved a lower validation MAE (0.07204 vs. 0.07623 and 0.07829).

This result suggests that the dataset has very little position-independent local structure (or shift-invariant patterns) for a convolutional layer to exploit effectively.

Convolutional layers work best when key patterns look the same regardless of where they occur in a sequence—similar to finding an edge or shape in an image. However, electricity price and load data are heavily tied to specific points in time, such as exact hours of the day or days of the week. Because adding a CNN layer before the LSTM actually increased the error, it shows that attempting to filter the raw input into localized, position-independent features strips away important temporal context rather than helping the model make better forecasts.

D2. Did hyperparameter tuning close the gap (or extend the lead) between the CNN-LSTM and the plain LSTM? State the direction and size of the change.

Hyperparameter tuning extended the lead of the plain LSTM by making the CNN-LSTM's performance worse rather than closing the gap. When looking at the baseline CNN-LSTM with 11,969 parameters, its validation loss was 0.00902 and its validation MAE was 0.07623, placing it behind the plain LSTM, which achieved a validation loss of 0.00807 and a validation MAE of 0.07204. After tuning the CNN-LSTM—which increased its parameter count to 16,113—its validation loss worsened to 0.00916 and its validation MAE increased to 0.07829.

In terms of the size of this change, the performance gap between the two architectures expanded across both metrics. The gap in validation loss grew by 0.00014, widening the difference between the models from 0.00095 to 0.00109. Similarly, the gap in validation MAE increased by 0.00206, widening from an initial difference of 0.00419 to 0.00625. Overall, tuning the hyperparameters did not help the CNN-LSTM recover its disadvantage, but instead allowed the plain LSTM to solidify its superior performance on this dataset.

D3. Looking at your tuner's `results_summary()` across its top 3 trials, which single hyperparameter appeared most consistently favorable (e.g. did most top trials prefer a particular kernel size, a small or large dropout rate)?

Looking at the architecture summaries, the hyperparameter that appears most consistently across the CNN-LSTM models is the use of 32 filters (or channels) in the convolutional layers. Both the baseline CNN-LSTM and the deeper variant retain an output depth of 32 feature maps ((None, 24, 32) and (None, 8, 32)), whereas scaling up to 64 filters in the tuned variant (`sequential_1`) significantly increased the total parameter count to over 62,000. This shows that keeping a compact layer width of 32 filters provided a more balanced capacity without drastically over-parameterizing the model.